

1. Заголовок

Кейс «Модель для обнаружения аномалий в числовых последовательностях» — направлен на разработку программного модуля для автоматического выявления выбросов в данных.

Направление подготовки: 09.03.03 – Прикладная информатика (или аналогичное)

Образовательная программа: Прикладной искусственный интеллект

Семестр: 2-й семестр

2. Аннотация

Кейс «Модель для обнаружения аномалий в числовых последовательностях» представляет собой задачу по созданию аналитического инструмента, способного находить отклонения в синтетических данных (временных рядах).

Учебная дисциплина: Проектный практикум / Введение в анализ данных

Актуальность и практическая значимость кейса обусловлена необходимостью автоматического мониторинга систем. Обнаружение аномалий (Anomaly Detection) используется повсеместно: от выявления мошенничества с банковскими картами до предсказания поломок оборудования на заводах по показаниям датчиков. Студенты научатся генерировать данные, применять базовые статистические методы и алгоритмы машинного обучения, а также визуализировать результаты. Решение кейса — это прототип системы мониторинга, который можно включить в портфолио начинающего Data Scientist'a.

Форма выполнения: групповая.

Кейс направлен на формирование следующих ключевых компетенций:

Профессиональные компетенции (hard skills):

- **Python для анализа данных:** Уверенное владение библиотеками NumPy, Pandas для обработки числовых последовательностей.
- **Алгоритмика и статистика:** Понимание базовых статистических методов (среднее, дисперсия, правило 3-х сигм) и применение готовых алгоритмов ML (например, Isolation Forest из Scikit-learn).
- **Визуализация данных:** Построение графиков (Matplotlib/Seaborn/Plotly) для наглядного отображения исходного сигнала и найденных аномалий.
- **Работа с синтетическими данными:** Навык генерации данных с заданными характеристиками (шум, тренд, сезонность, выбросы).
- **Разработка MVP интерфейса:** Создание простого веб-интерфейса для демонстрации работы модели (рекомендуется использовать Streamlit или Dash, чтобы избежать сложной frontend-разработки).

Универсальные компетенции (soft skills):

- **Командная работа:** Распределение ролей (генерация данных, алгоритмы, интерфейс) и синхронизация кода.
- **Работа с инструментами разработки:** Использование отечественных платформ Сфера.Задачи (трекинг), Сфера.Знания (документация), Сфера.Код (контроль версий).
- **Аналитическое мышление:** Умение формализовать понятие «аномалия» и выбрать критерии качества работы алгоритма.

3. Постановка задания

Название кейса: Разработка инструмента для детекции аномалий (MVP).

Цель выполнения кейса:

Развитие навыков работы с данными, реализации математических алгоритмов в коде и создания демонстрационных прототипов. Студенты должны пройти путь от генерации данных до визуализации найденных отклонений, работая в команде по методологии Agile.

Конкретный образ результата:

Рабочее приложение (локально или на хостинге), написанное на Python (рекомендуется Streamlit), которое позволяет:

1. Генерировать числовую последовательность (временной ряд) с заданными параметрами (например: синусоида с шумом + случайные выбросы).
2. Выбирать алгоритм поиска аномалий (статистический или ML).
3. Отображать график, где цветом выделены найденные аномальные точки.
4. Выводить базовые метрики (сколько аномалий найдено).

Промежуточные показы (Milestones):

В течение проекта предусмотрено 4 обязательных промежуточных показа.

- **Показ 1 (Конец 3-й недели):** Защита плана и генератора данных.
- **Показ 2 (Конец 6-й недели):** Реализация статистического метода поиска (Baseline).
- **Показ 3 (Конец 9-й недели):** Реализация ML-метода (использование библиотеки).
- **Показ 4 (Конец 12-й недели):** Демонстрация интерфейса с визуализацией.

Форма и формат отчетности (финальная):

1. Ссылка на репозиторий (Сфера.Код) с исходным кодом.
2. Демонстрация работы приложения (скринкаст или живой показ).
3. Техническая документация (в Сфера.Знания или README), описывающая, как запускать проект и какие алгоритмы использованы.
4. Финальная презентация (защита решения).

Сроки выполнения: 15 недель.

Краткие сведения о системе оценивания:

Накопительная система. Оценивается своевременность выполнения спринтов, чистота кода, корректность работы алгоритмов на сгенерированных данных и понятность визуализации.

4. Рекомендуемые инструменты

Таск-трекер Сфера.Задачи для настройки доски, постановки задач с ролями по спринтам и фиксации дефектов.

Wiki-система Сфера.Знания для формирования и хранения технической документации.

Git-репозиторий Сфера.Код для создания, версионирования и хранения кода.

5. Методические рекомендации обучающимся

Рекомендуемый состав группы по ролям (для команды из 3-4 человек):

- **Team Lead / Аналитик:** Организация работы в Сфера.Задачи, определение критериев «что такое аномалия» для текущей задачи, подготовка презентации.
- **Data Engineer (Генерация данных):** Написание скриптов для создания синтетических данных (нормальное поведение + подмешивание аномалий).
- **ML Engineer (Алгоритмы):** Реализация функций поиска аномалий (сначала простых, потом сложнее).
- **Python Dev (Визуализация):** Сборка итогового интерфейса на Streamlit, интеграция кода коллег.

Примечание: Роли могут пересекаться, студенты могут писать код совместно.

Примерный план работы по спринтам (3 недели на спринт):

Спринт 1 (Недели 1-3): Данные и Исследование

- **Задачи:** Распределить роли. Изучить библиотеки (NumPy, Matplotlib). Написать скрипт генерации синтетических данных: например, ровная линия с шумом, синусоида. Добавить функцию внесения случайных «выбросов» (резких скачков значений). Настроить репозиторий в Сфера.Код.

● **Результат для Показа 1:** Демонстрация скрипта, который генерирует данные и рисует график, где глазами видны искусственно созданные аномалии.

Спринт 2 (Недели 4-6): Статистический подход (Baseline)

- **Задачи:** Реализовать простейший алгоритм поиска аномалий. Например, метод «3-х сигм» (Z-score) или межквартильный размах (IQR). Алгоритм должен принимать массив данных и возвращать индексы аномальных точек.

- **Результат для Показа 2:** Демонстрация работы алгоритма в Jupyter Notebook или консоли: на вход подаются данные из Спринта 1, на выходе — список найденных аномалий. Сравнение найденных с реально добавленными.

Спринт 3 (Недели 7-9): Продвинутый подход (ML)

- **Задачи:** Подключить библиотеку Scikit-learn. Использовать готовый алгоритм, например, Isolation Forest или One-Class SVM. Настроить его параметры для работы с вашими данными.

- **Результат для Показа 3:** Сравнение работы статистического метода (Спринт 2) и ML-метода. Какой лучше находит скрытые аномалии?

Спринт 4 (Недели 10-12): Интерфейс и Визуализация

- **Задачи:** Создать простое веб-приложение (используя библиотеку Streamlit — это позволяет создавать UI на чистом Python без HTML/JS). Добавить кнопки: «Сгенерировать новые данные», «Найти аномалии методом А», «Найти методом Б». Вывести интерактивный график.

- **Результат для Показа 4:** Работающий прототип. Пользователь нажимает кнопки в браузере и видит результат анализа.

Спринт 5 (Недели 13-15): Доработка и Подготовка к защите

- **Задачи:** Исправление ошибок. Оформление кода (комментарии, структура). Написание документации в Сфера.Знания (или README). Подготовка слайдов: описание задачи, выбранных алгоритмов и примеров работы.

- **Результат:** Готовый проект и успешная защита.

6. Методические рекомендации по оцениванию

Общая система оценивания: 100-балльная шкала.

Критерии оценивания (максимум 100 баллов):

Блок 1. Промежуточные показы (40 баллов)

- **Показ 1 (10 баллов):** Данные генерируются корректно, код выложен в репозиторий. Команда понимает задачу.

- **Показ 2 (10 баллов):** Реализован статистический метод. Он находит явные выбросы.

- **Показ 3 (10 баллов):** Реализован и запущен метод из ML-библиотеки.

- **Показ 4 (10 баллов):** Есть рабочий интерфейс, графики строятся.

Блок 2. Качество реализации (40 баллов)

- **Корректность алгоритмов (15 баллов):** Алгоритмы действительно находят аномалии, параметры подобраны адекватно (не выделяют всё подряд как аномалию).

- **Качество кода (15 баллов):** Код структурирован (разбит на функции/файлы), переменные названы понятно. Использован Git (есть история коммитов).

- **Визуализация (10 баллов):** Графики понятные, легенда присутствует, аномалии выделены цветом.

Блок 3. Документация и защита (20 баллов)

- **Документация (5 баллов):** Понятная инструкция по запуску.

- **Презентация (10 баллов):** Четкое изложение: какая была задача, какие данные, какой метод выбрали и почему.

- **Ответы на вопросы (5 баллов):** Понимание того, как работают использованные алгоритмы (на базовом уровне).

Описание уровней оценки:

- **Отлично (80-100):** Все работает, интерфейс удобный, использовано два разных алгоритма для сравнения. Студенты понимают математику (на уровне 2 семестра) примененных методов.

- **Хорошо (60-79):** Приложение работает, но есть мелкие баги или интерфейс неудобен. Реализован только один алгоритм хорошо.
- **Удовлетворительно (40-59):** Приложение запускается с трудом. Реализован только простейший статистический метод. Визуализация слабая.
- **Неудовлетворительно (0-39):** Проект не работает или отсутствует код. Нет понимания задачи.